



The Vault Unstaker V2

SECURITY ASSESSMENT REPORT

November 18, 2025

Prepared for:





Contents

1	About CODESPECT	2
2	Disclaimer	2
3	Risk Classification	3
4	Executive Summary	4
5	Audit Summary	5
5.1	Scope - Audited Files	5
5.2	Findings Overview	6
6	System Overview	7
7	Issues	10
7.1	[Critical] The <code>flash_borrow(...)</code> instruction can be used to drain the vault	10
7.2	[High] The <code>withdraw_stake_account(...)</code> instruction does not assign the <code>StakeAccount</code> authority to the user	10
7.3	[Medium] The <code>withdraw_stake_account(...)</code> instruction may dilute vault rewards	11
7.4	[Low] Inconsistent rent refund	11
7.5	[Low] The <code>liquid_unstake_lst(...)</code> and <code>liquid_unstake_lst_wrapped(...)</code> instructions lack address verification.	12
7.6	[Info] Inconsistent flash loan validation	12
7.7	[Info] Some instructions could be DoS due to account creation failure	13
7.8	[Best Practice] Unsafe to close <code>stake_account_info_pda</code>	14
8	Evaluation of Provided Documentation	15
9	Test Suite Evaluation	16
9.1	Compilation Output	16
9.2	Tests Output	16
9.3	Notes on the Test Suite	17



1 About CODESPECT

CODESPECT is a specialized smart contract security firm dedicated to ensure the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to build secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialize in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralized applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialized cybersecurity solutions designed to minimize risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty program to incentivize external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivized.
- **Medium** - Likely only under certain conditions or moderately incentivized.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilizes two other categories for findings: **Informational** and **Best Practices**.

- a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally.
- b) **Best Practices** findings indicate that certain portions of the code deviate from established smart contract development standards.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for The Vault Unstaker program of The Vault. The Vault protocol suite provides a set of on-chain programs related to Solana staking functionality.

This audit focuses on the Unstaker program, which allows users to unstake their LST immediately in exchange for a dynamic fee.

The audit was performed using:

- a) Manual analysis of the codebase.
- b) Dynamic analysis of smart contracts, execution testing.

CODESPECT found eight points of attention, one classified as Critical, one classified as High, one classified as Medium, two classified as Low, two classified as Informational, and one classified as Best Practices. All of the issues are summarised in Table 2.

Organisation of the document is as follows:

- **Section 5** summarizes the audit.
- **Section 6** describes the functionality of the code in scope.
- **Section 7** presents the issues.
- **Section 8** discusses the documentation provided by the client for this audit.
- **Section 9** presents the compilation and tests.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
Critical	0	1	0
High	0	1	0
Medium	0	1	0
Low	0	2	0
Informational	0	2	0
Best Practices	0	1	0
Total	0	8	0

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

Audit Type	Security Review
Project Name	The Vault
Type of Project	Vault for unstaking
Duration of Engagement	7 Days
Duration of Fix Review Phase	2 Days
Draft Report	November 13, 2025
Final Report	November 17, 2025
Repository	liquid-unstaker
Commit (Audit)	2dd71f5e3d9017504783c67bc1b903ecaa169a54
Commit (Final)	ade63ba7116618dda6ad9a54327bfcd899edd813
Documentation Assessment	Medium
Test Suite Assessment	Medium
Auditors	jecikPo , shaflow01

Table 3: Summary of the Audit

5.1 Scope - Audited Files

	Contract	LoC
1	instructions/liquid_unstake_lst_wrapped.rs	209
2	instructions/withdraw_stake_account.rs	208
3	instructions/create_or_update_token_metadata.rs	196
4	instructions/liquid_unstake_lst.rs	166
5	instructions/flash_borrow.rs	147
6	instructions/liquid_unstake_stake_account.rs	127
7	instructions/update.rs	114
8	fee.rs	107
9	lp_math.rs	107
10	lib.rs	101
11	instructions/flash_repay.rs	93
12	error.rs	90
13	instructions/deposit_sol.rs	85
14	instructions/initialize_pool.rs	80
15	state.rs	79
16	instructions/withdraw_sol.rs	69
17	fee_flash_loans.rs	55
18	instructions/update_pool.rs	49
19	instructions/mod.rs	24
20	fee_lp.rs	23
21	seeds.rs	6
	Total	2135



5.2 Findings Overview

	Finding	Severity	Update
1	The <code>flash_borrow(...)</code> instruction can be used to drain the vault	Critical	Fixed
2	The <code>withdraw_stake_account(...)</code> instruction does not assign the <code>StakeAccount</code> authority to the user	High	Fixed
3	The <code>withdraw_stake_account(...)</code> instruction may dilute vault rewards	Medium	Fixed
4	Inconsistent rent refund	Low	Fixed
5	The <code>liquid_unstake_1st(...)</code> and <code>liquid_unstake_1st_wrapped(...)</code> instructions lack address verification.	Low	Fixed
6	Inconsistent flash loan validation	Info	Fixed
7	Some instructions could be DoS due to account creation failure	Info	Fixed
8	Unsafe to close <code>stake_account_info_pda</code>	Best Practices	Fixed



6 System Overview

The **Liquid Unstaker** protocol facilitates instant unstaking for users holding Liquid Staking Tokens (LSTs) or native SOL stake accounts, while allowing liquidity providers to supply the necessary SOL and earn fees in return.

The protocol operates through **liquidity pools** where:

- **Liquidity Providers (LPs)** deposit SOL and receive LP tokens representing their share of the pool,
- **Users** can instantly unstake LSTs or SOL stake accounts and pay fees,
- **Flash loan borrowers** can access pool liquidity within a single transaction for arbitrage or other purposes,
- The system manages deactivating stake accounts and returns SOL to the pool as it becomes available.

Key Flows

The following subsections describe key flows in detail.

Pool Initialization

Each pool is created via the `initialize_pool` instruction with the following configurable parameters:

- **Fee Structure:**
 - `fee_max` — Maximum unstaking fee
 - `fee_min` — Minimum unstaking fee
 - `min_sol_for_min_fee` — SOL threshold where minimum fee applies
 - `manager_fee_pct` — Manager's share of fees
- **Liquidity Controls:**
 - `vault_lamports_cap` — Maximum SOL deposit limit
- **LP Withdrawal Fees:**
 - `withdraw_sol_fee` — Fee when LPs withdraw SOL
 - `withdraw_stake_account_fee` — Fee when LPs withdraw as stake account
- **Flash Loan Configuration:**
 - `flash_loans_enabled` — Toggle flash loan functionality
 - `flash_loan_fee` — Fee for flash loans

Pool configuration can be updated at any time by the pool authority using the `update_pool` instruction.

Depositing SOL

Liquidity providers deposit SOL into the pool and receive LP tokens in exchange using the `deposit_sol` instruction. The number of LP tokens minted is calculated based on:

- The current pool state (total SOL and total LP supply),
- **Epoch rewards vesting** — rewards from stake accounts are linearly vested over the epoch to prevent deposit/withdraw gaming,

Deposits are subject to the `vault_lamports_cap` limit to prevent excessive pool size.

Withdrawing SOL

Liquidity providers can burn their LP tokens to withdraw SOL using the `withdraw_sol` instruction. The withdrawal process:

- Calculates SOL amount based on LP share and vested rewards,
- Deducts the `withdraw_sol_fee` (sent to manager fee account),
- Burns the LP tokens and transfers SOL to the provider,



Withdrawing Stake Account

As an alternative to SOL withdrawal, LPs can receive a deactivating stake account using the `withdraw_stake_account` instruction.:

This mechanism:

- Uses the stake program's `split` instruction to create a new stake account,
- Transfers the deactivating stake to the LP,
- Applies the `withdraw_stake_account_fee`,

Unstaking LSTs

Users holding LST tokens from SPL Stake Pools can instantly convert them to SOL using the `liquid_unstake_1st` instruction:

The unstaking flow:

- a. The instruction interacts with the SPL Stake Pool program to withdraw stake,
- b. Supports up to 5 validator stake accounts per transaction,
- c. Deactivates the received stake account(s) immediately,
- d. Transfers SOL (minus dynamic fees) from the vault to the user,
- e. Creates a `StakeAccountInfo` PDA to track the deactivating stake.

Variant: PDA Stake Account Creation

The `liquid_unstake_1st_with_seed` variant creates stake accounts as PDAs using a deterministic seed, enabling programmatic integration without keypair management.

Variant: Wrapped LST Support

The protocol includes `liquid_unstake_1st_with_wrapped` and `liquid_unstake_1st_with_wrapped_seed` variants for wrapped LST tokens, with adjusted fee and rent handling.

Unstaking fees are dynamically calculated based on the pool's liquidity to incentivize LP deposits and protect the pool.

Epoch Rewards Vesting

To prevent gaming through deposit-withdraw arbitrage, the protocol implements **linear vesting** of stake rewards over each epoch:

- Rewards from deactivating stake accounts are tracked separately when withdrawn via the `update` instruction,
- These rewards vest linearly throughout the epoch based on slot progression,
- LP token calculations (both mint and burn) use only vested rewards in the total value,
- Resets at epoch boundaries to begin vesting the next epoch's rewards.

This mechanism ensures fair value distribution and prevents users from depositing just before reward collection and immediately withdrawing.

Flash Loans

The protocol includes a flash loan system allowing users to borrow SOL from the vault within a single transaction, this is done using `flash_borrow` and `flash_repay` instructions.

Borrowing

The borrowing process:

- Validates that `flash_loans_enabled` is true,
- Checks available liquidity (only vault SOL, excluding deactivating stake),
- Uses **instruction introspection** to verify a corresponding `flash_repay` exists in the same transaction,
- Transfers SOL from vault to borrower,
- Records the borrowed amount in the pool state,
- Cannot be called via CPI — security measure to prevent malicious integrations.



Repayment

The repayment process:

- Must occur in the same transaction as `flash_borrow`,
- Calculates the flash loan fee based on borrowed amount,
- Splits the fee between the pool and the manager
- Validates full repayment (principal + fee),
- Resets `flash_loan_borrowed_amount` to zero,
- Cannot be called via CPI — prevents exploits.

Flash loan fees are capped and can be configured per pool.



7 Issues

7.1 [Critical] The `flash_borrow(...)` instruction can be used to drain the vault

File(s): `state.rs`

Description: The `flash_borrow(...)` instruction allows users to perform flash loans using funds from the vault. The user is required to include a `flash_repay(...)` instruction later in the same transaction to repay the loan. During the flash loan, the borrowed amount is recorded in `flash_loan_borrowed_amount`, and `sol_vault_lamports` decreases accordingly. However, when calculating the vault's share price, only `sol_vault_lamports` is considered, while `flash_loan_borrowed_amount` is ignored. This causes the share value to drop temporarily during the flash loan.

```
pub fn flash_borrow(&mut self, amount: u64) -> Result<> {
    self.flash_loan_borrowed_amount = amount;
    self.sol_vault_lamports = self.sol_vault_lamports
        .checked_sub(amount)
        .ok_or(error!(crate::error::LiquidUnstakerErrorCode::InsufficientSolVaultBalance))?;
    Ok(())
}
```

Impact: An attacker can exploit this mechanism to drain the vault by performing an arbitrage: they first initiate a flash loan to reduce the share value, and then mint shares at the temporarily low price. After repaying the flash loan, `sol_vault_lamports` returns to normal, causing the share value to rise again. Then the attacker can burn their shares for profit.

Recommendation(s): It is recommended to prohibit interactions with the vault during the flash loan process.

Status: Fixed

Update from The Vault: Resolved in [140cc52e5bc519fc9a1c223987c8d8f5d0c001ac](#) and [9aeeaa21ee6efed2524d0f32dbef306c103697f2](#)

7.2 [High] The `withdraw_stake_account(...)` instruction does not assign the `StakeAccount` authority to the user

File(s): `withdraw_stake_account.rs`

Description: The `withdraw_stake_account(...)` instruction allows users to burn shares even when the vault has insufficient available funds. It splits a `StakeAccount` that has not yet been deactivated into a new `StakeAccount` containing the corresponding amount of SOL, enabling an immediate withdrawal. However, after splitting the `StakeAccount` via `stake_split(...)`, the authority of the newly created `StakeAccount` is not assigned to the user, preventing them from operating the split account.

Impact: The user burns their shares through `withdraw_stake_account(...)` but does not receive any accessible funds, and the program is also unable to operate on those funds. As a result, this portion of funds becomes stuck.

Recommendation(s): It is recommended to assign the authority of the `StakeAccount` split through `stake_split(...)` to the user.

Status: Fixed

Update from The Vault: Resolved in [e3a457993390529b803800375a3b6b493e975e5a](#)



7.3 [Medium] The `withdraw_stake_account(...)` instruction may dilute vault rewards

File(s): `withdraw_stake_account.rs`

Description: The `withdraw_stake_account(...)` instruction allows users to withdraw funds by splitting the `stake_account` using the `split(...)` instruction before the `stake_account` is deactivated. However, since the `stake_account` may earn staking rewards at the end of the epoch, these rewards would normally be allocated to the vault after the update instruction. Allowing the `stake_account` to be split early could cause rewards that were originally meant for the vault to be allocated to the withdrawing user instead. At the same time, when `stake_account_info_source.stake_lamports` equals 0, `stake_account_info_source` will be closed without checking whether there are still funds in the `stake_account` (e.g., donated lamports). This could result in some funds in the `stake_account` being stuck or if the amount left is smaller than `minimum_delegation_amount` the instruction will fail.

```
if ctx.accounts.stake_account_info_source.stake_lamports == 0 {
    // Close the PDA if all stake has been withdrawn (and remaining lamports transferred to vault)
    let before_vault_lamports = ctx.accounts.sol_vault.lamports();
    ctx.accounts.stake_account_info_source.close(ctx.accounts.sol_vault.to_account_info());
    let after_vault_lamports = ctx.accounts.sol_vault.lamports();
    // Update vault lamports in pool accounting
    ctx.accounts.pool.sol_vault_lamports = ctx.accounts.pool.sol_vault_lamports
        .checked_add(after_vault_lamports)
        .checked_sub(before_vault_lamports)
        .ok_or(LiquidUnstakerErrorCode::MathUnderflow)?;
    .ok_or(LiquidUnstakerErrorCode::MathOverflow)?;
}
```

Impact: If the `stake_account` has rewards, prematurely splitting the `stake_account` to exit the vault would benefit the withdrawer and cause the vault to lose rewards. The second impact is that when a malicious user donates less than `minimum_delegation_amount` the `split(...)` CPI will fail preventing successful full withdrawals.

Recommendation(s): When `ctx.accounts.stake_account_info_source.stake_lamports = 0`, only close it if the `stake_account` has no lamports. Solving the reward dilution issue is not easy, so it is recommended to consider whether to keep the `withdraw_stake_account(...)` instruction.

Status: Fixed

Update from The Vault: Resolved in [575250e813947cd86a83bdcdb8fe23c02ebd67e6](#) and [ade63ba7116618dda6ad9a54327bfcd899edd813](#)

7.4 [Low] Inconsistent rent refund

File(s): `liquid_unstake_stake_account.rs`

Description: The `StakeAccount` authority and the LST holder can interact with the vault to assign a deactivating account to the pool and immediately withdraw SOL from the vault. This process requires the user to pay rent to create the `StakeAccountInfo` account. In the `liquid_unstake_stake_account(...)` instruction, the rent is not refunded. When the account is released, the rent goes to the vault. However, in the `liquid_unstake_lst(...)` and `liquid_unstake_lst_wrapped(...)` instructions, the vault reimburses the payer for the rent before the instruction ends.

Impact: The inconsistent refund logic may affect accounting consistency, predictability, and user experience.

Recommendation(s): It is recommended to standardize whether rent should be refunded.

Status: Fixed

Update from The Vault: Resolved in [00e927a9a3ceef5c3a4a4dccffefbb756c5f8216](#)

7.5 [Low] The `liquid_unstake_1st(...)` and `liquid_unstake_1st_wrapped(...)` instructions lack address verification.

File(s): `liquid_unstake_1st.rs`, `liquid_unstake_1st_wrapped.rs`

Description: In the `liquid_unstake_1st(...)` and `liquid_unstake_1st_wrapped(...)` instructions, a `stake_account_info` account is created. The expected account address is `b"stake_account_info"`, a PDA derived using `destination_stake_accounts` as the seed. However, there is no account address verification, which allows a malicious actor to provide any signed address to be created as `stake_account_info` instead of the intended address.

```
let create_stake_account_info_instruction = solana_program::system_instruction::create_account(
    &ctx.accounts.payer.key(),
    &stake_info_accounts[i].key(),
    stake_info_rent,
    8 + StakeAccountInfo::LEN as u64,
    &ctx.program_id
);
let (_, bump) = Pubkey::find_program_address(&b"stake_account_info", destination_stake_accounts[i].key.as_ref()),
    ↪ &crate::id());

solana_program::program::invoke_signed(
    &create_stake_account_info_instruction,
    &[
        ctx.accounts.payer.to_account_info(),
        stake_info_accounts[i].to_account_info(),
        ctx.accounts.system_program.to_account_info(),
    ],
    &[&b"stake_account_info", destination_stake_accounts[i].key.as_ref(), &bump]],
)?;
```

Impact: If the above situation occurs, the `withdraw_stake_account(...)` instruction will fail due to seed mismatch, but the funds can still eventually be released to the vault via the `update(...)` instruction.

Recommendation(s): It is recommended to verify that the `stake_info_accounts` passed in match the expected addresses.

```
- let (_, bump) = Pubkey::find_program_address(&b"stake_account_info",
- destination_stake_accounts[i].key.as_ref()), &crate::id());
+ let (expect_address, bump) = Pubkey::find_program_address(&b"stake_account_info",
+ destination_stake_accounts[i].key.as_ref()), &crate::id());
+ if stake_info_accounts[i].key != expect_address {
+     return Err(ProgramError::InvalidAccountData);
+ }
```

Status: Fixed

Update from The Vault: Resolved in [d9b62dc9c110a2f8693e000ce2eb51dff5c45430](#)

7.6 [Info] Inconsistent flash loan validation

File(s): `flash_borrow.rs`

Description: The `borrow(...)` instructions contains the `verify_flash_repay_instruction_exists(...)` function which validates the `repay(...)` instruction which must be present within the same transaction. One of the validations ensures that the instruction argument of the repayment account is enough to repay the whole loan including fees:

```
require!(
    repay_amount >= minimum_repayment,
    LiquidUnstakerErrorCode::FlashLoanRepaymentAmountInsufficient
);
```

A similar validation is present within the `repay` instruction:

```
require!(
    repay_amount == expected_repayment,
    LiquidUnstakerErrorCode::FlashLoanRepaymentAmountInsufficient
);
```



Yet, with a `==` instead of `>=`, which is inconsistent. The borrow validation is less strict than repay, hence it could pass, but the repay would fail.

Impact: No security impact. The code would revert later than expected.

Recommendation(s): Implement stricter amount validation on the borrow side with `==`.

Status: Fixed

Update from The Vault: [ef7a1e4670d835d98592d2feb4142cb94595e5e9](#)

7.7 [Info] Some instructions could be DoS due to account creation failure

File(s): `withdraw_stake_account.rs`, `liquid_unstake_lst_wrapped.rs`, `liquid_unstake_lst.rs`

Description: Some instructions need to create new accounts during execution, and due to improper checks, they could be DoS'ed. In the `liquid_unstake_lst_wrapped(...)` and `withdraw_stake_account(...)` instructions, when creating a `stake_account`, the account creation is skipped as long as the target account's lamports are not 0. A malicious actor can front-run by depositing some lamports into the account, causing the instruction to fail.

```
// Create the destination stake account if not created already (paid by user)
if ctx.accounts.stake_account_destination.lamports() == 0 {

    let rent = rent.minimum_balance(solana_program::stake::state::StakeStateV2::size_of());

    let create_stake_account_info_instruction = solana_program::system_instruction::create_account(
        &ctx.accounts.user.key(),
        &ctx.accounts.stake_account_destination.key(),
        rent,
        solana_program::stake::state::StakeStateV2::size_of() as u64,
        &solana_program::stake::program::id()
    );
    //...
}
```

In the `liquid_unstake_lst_wrapped(...)` and `liquid_unstake_lst(...)` instructions, `stake_account_info` is created using `create_account`. Since the `create_account` instruction fails if the target account already has lamports, a malicious actor can front-run by depositing some lamports into the account, causing the `create_account` call to fail.

```
// Create the stake account info account
let stake_info_rent = Rent::get()?.minimum_balance(8 + StakeAccountInfo::LEN);
accumulated_rent_pda_accounts += stake_info_rent;

let create_stake_account_info_instruction = solana_program::system_instruction::create_account(
    &ctx.accounts.payer.key(),
    &stake_info_accounts[i].key(),
    stake_info_rent,
    8 + StakeAccountInfo::LEN as u64,
    &ctx.program_id
);
```

Impact: Some instruction calls could be DoS if a malicious actor donates lamports.

Recommendation(s): When checking whether an account needs to be created, verify not only whether lamports exist but also whether the account data size matches the expected size. When creating the account, call `create_account` only if lamports are 0; if not, top up the missing rent and then use `system_instruction::allocate` and `system_instruction::assign` to create the account.

Status: Fixed

Update from The Vault: Resolved in [7f5463199bc1da21df0f9e232e39502b10f4cfb](#)



7.8 [Best Practice] Unsafe to close stake_account_info_pda

File(s): [update.rs](#)

Description: In the `update(...)` instruction, lamports are withdrawn from the `stake_account`, and the related `stake_account_info_pda` account is closed.

```
for stake_account_info_pda in stake_account_info_pda_remaining_accounts {  
    total_rent_amount += stake_account_info_pda.get_lamports();  
    **stake_account_info_pda.try_borrow_mut_lamports()? = 0;  
}
```

However, the method for closing `stake_account_info_pda` is to transfer out all lamports, and the data clearing occurs after the transaction ends. A malicious caller of the `update(...)` instruction could provide lamports to `stake_account_info_pda` before the transaction ends, preventing the account from being closed.

Impact: If the closed `stake_account` is re-initialized as a `stake_account`, it cannot interact with the vault because the `stake_account_info_pda` was not closed.

Recommendation(s): It is recommended to deterministically set the `stake_account_info_pda` space to 0.

Status: Fixed

Update from The Vault: [472730d353adca49fab795f1c5c20c3c42bcada5](#)



8 Evaluation of Provided Documentation

The **Liquid Unstaker** documentation was provided in the form of a README file and NatSpec comments:

- **NatSpec:** The in-code NatSpec comments were generally sufficient and very helpful in explaining specific flows and code branches. In several cases, they also clarified assumptions underlying the implementation, providing context for why certain approaches were taken and the intended behavior of the system.
- **README:** The provided README offered a rather basic overview of the protocol's functionality.

Overall, the documentation was adequate for the scope of this audit, however it could be further improved, by documenting specific token flows, including different fees and rent refunds. Yet The Vault team remained consistently available and responsive, promptly addressing all questions and concerns raised by **CODESPECT** throughout the audit process.

9 Test Suite Evaluation

9.1 Compilation Output

```
% cargo build
Compiling liquid-unstaker v0.1.0 (040-Vault-Liquid-Unstaker/programs/liquid-unstaker)
[...]
Finished `dev` profile [unoptimized + debuginfo] target(s) in 1m 03s
```

9.2 Tests Output

```
% cargo test
[...]
Compiling liquid-unstaker v0.1.0 (040-Vault-Liquid-Unstaker/programs/liquid-unstaker)
warning: `liquid-unstaker` (lib) generated 5 warnings (run `cargo fix --lib -p liquid-unstaker` to apply 1 suggestion)
warning: `liquid-unstaker` (lib test) generated 4 warnings (4 duplicates)
Finished `test` profile [unoptimized + debuginfo] target(s) in 2.03s
Running unittests src/lib.rs (target/debug/deps/liquid_unstaker-d4ccd121445d3e1c)

running 88 tests
test fee::tests::fee_large_unstake_below_min_sol_for_min_fee ... ok
test fee::tests::test_calculate_base_fee_exactly_at_threshold ... ok
test fee::tests::linear_fee_when_vault_has_less_than_min_sol_for_min_fee ... ok
test fee::tests::test_calculate_base_fee_edge_underflow ... ok
test fee::tests::fee_min_when_vault_has_more_than_min_sol_for_min_fee ... ok
test fee::tests::test_calculate_base_fee_large_numbers ... ok
test fee::tests::test_calculate_base_fee_max_values ... ok
test fee::tests::test_calculate_base_fee_vault_at_zero ... ok
test fee::tests::test_calculate_base_fee_underflow ... ok
test fee::tests::test_calculate_for_two_unstakes ... ok
test fee::tests::test_calculate_base_fee_zero_unstake ... ok
test fee::tests::test_calculate_linear_base_fee_edge_cases ... ok
test fee::tests::test_calculate_linear_base_fee_overflow ... ok
test fee::tests::test_calculate_linear_fee ... ok
test fee::tests::test_total_fee ... ok
test fee::tests::test_validate_fees_boundary ... ok
test fee::tests::test_validate_fees_edge_cases ... ok
test fee::tests::test_validate_fees_manager_pct_too_high ... ok
test fee::tests::test_validate_fees_max_less_than_min ... ok
test fee::tests::test_validate_fees_max_too_high ... ok
test fee::tests::test_validate_fees_valid ... ok
test fee_flash_loans::tests::test_calculate_flash_loan_fees ... ok
test fee_flash_loans::tests::test_calculate_flash_loan_fees_all_manager_fee ... ok
test fee_flash_loans::tests::test_calculate_flash_loan_fees_complex_split ... ok
test fee_flash_loans::tests::test_calculate_flash_loan_fees_no_manager_fee ... ok
test fee_flash_loans::tests::test_calculate_flash_loan_fees_rounding ... ok
test fee_flash_loans::tests::test_calculate_flash_loan_fees_zero_flash_loan_fee ... ok
test fee_flash_loans::tests::test_calculate_flash_loan_total_fee ... ok
test fee_flash_loans::tests::test_calculate_flash_loan_total_fee_high_precision ... ok
test fee_flash_loans::tests::test_calculate_flash_loan_total_fee_max_fee ... ok
test fee_flash_loans::tests::test_calculate_flash_loan_total_fee_no_fee ... ok
test fee_flash_loans::tests::test_flash_loan_fee_total_fee ... ok
test fee_flash_loans::tests::test_flash_loan_fee_total_fee_overflow ... ok
test fee_flash_loans::tests::test_new_flash_loan_fee ... ok
test fee_flash_loans::tests::test_new_flash_loan_fee_invalid ... ok
test fee_flash_loans::tests::test_validate_flash_loan_fee ... ok
test fee_lp::tests::test_calculate_withdraw_fee ... ok
test fee_lp::tests::test_calculate_withdraw_fee_max_fee ... ok
test fee_lp::tests::test_calculate_withdraw_fee_no_fee ... ok
test fee_lp::tests::test_validate_withdraw_fee ... ok
test instructions::flash_borrow::discriminator_tests::test_flash_repay_discriminator ... ok
test lp_math::tests::test_burn_empty_pool ... ok
test lp_math::tests::test_burn_entire_pool ... ok
test lp_math::tests::test_burn_entire_pool_with_fee ... ok
test lp_math::tests::test_burn_non_empty_pool ... ok
test lp_math::tests::test_burn_overflow_protection ... ok
test lp_math::tests::test_burn_precision_rounding ... ok
test lp_math::tests::test_burn_small_amount ... ok
test lp_math::tests::test_burn_with_fee ... ok
```

```

test lp_math::tests::test_burn_with_fee_precision ... ok
test lp_math::tests::test_burn_with_max_fee ... ok
test lp_math::tests::test_calculate_lamports_to_withdraw_with_epoch_rewards ... ok
test lp_math::tests::test_calculate_tokens_to_mint_with_epoch_rewards ... ok
test lp_math::tests::test_epoch_rewards_full_progress_reduces_pool ... ok
test lp_math::tests::test_epoch_rewards_no_impact_when_zero ... ok
test lp_math::tests::test_epoch_rewards_status_default ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_full_progress ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_half_progress ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_large_amounts ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_overflow_protection ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_precision ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_quarter_progress ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_small_progress ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_three_quarter_progress ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_underflow_protection ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_zero_progress ... ok
test lp_math::tests::test_epoch_rewards_status_lamports_rewards_available_zero_withdrawn ... ok
test lp_math::tests::test_epoch_rewards_status_linear_vesting_behavior ... ok
test lp_math::tests::test_mint_and_withdraw_symmetry_with_epoch_rewards ... ok
test lp_math::tests::test_mint_empty_pool ... ok
test lp_math::tests::test_mint_large_numbers ... ok
test lp_math::tests::test_mint_non_empty_pool ... ok
test lp_math::tests::test_mint_overflow ... ok
test lp_math::tests::test_mint_precision_rounding ... ok
test lp_math::tests::test_mint_with_half_epoch_progress ... ok
test lp_math::tests::test_mint_with_large_rewards_early_epoch ... ok
test lp_math::tests::test_mint_with_quarter_epoch_progress ... ok
test lp_math::tests::test_mint_with_small_deposit_large_rewards ... ok
test lp_math::tests::test_mint_with_three_quarter_epoch_progress ... ok
test lp_math::tests::test_mint_zero_deposit ... ok
test lp_math::tests::test_withdraw_entire_supply_with_epoch_rewards ... ok
test lp_math::tests::test_withdraw_small_amount_with_large_epoch_rewards ... ok
test lp_math::tests::test_withdraw_with_fee_and_epoch_rewards ... ok
test lp_math::tests::test_withdraw_with_half_epoch_progress ... ok
test lp_math::tests::test_withdraw_with_high_fee_and_epoch_rewards ... ok
test lp_math::tests::test_withdraw_with_quarter_epoch_progress ... ok
test lp_math::tests::test_withdraw_with_three_quarter_epoch_progress ... ok
test test_id ... ok

test result: ok. 88 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Doc-tests liquidunstaker

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

[...]
```

9.3 Notes on the Test Suite

The Liquid Unstaker test suite provides comprehensive coverage for the protocol's primary flows and addresses most common usage scenarios.

While the existing tests are valuable, the primary area for improvement is the end to end reward flow involving the Staking program and flash loan functionality. For the flash loan functionality it is key to verify the unstaker operations which are not permitted during a flash loan. The tests should also contain the stake account withdrawal functionality.